AFTER THE COURSE

Next steps

Learning path, certification and features to explore

Altkom Akademia
Edition: October 2026

Contents

PART 1	Learning path	3
PART 2	Certification	3
PART 3	Auto Loader	4
PART 4	Lakeflow Connect	5
PART 5	Lakeflow Spark Declarative Pipelines	5
PART 6	Unity Catalog in depth	7
PART 7	Structured Streaming	8
PART 8	Performance	9
PART 9	MLflow	10
PART 10	Advanced orchestration	11
PART 11	Integrations	12

Databricks Fundamental introduces the platform, Spark, Delta Lake, orchestration and Unity Catalog. This guide suggests what to learn next: a path through further courses and certification, and the platform features worth exploring once the basics are in place.

PART 1

Learning path

Stage	Course or credential	Scope
Foundation	Databricks Fundamental	Platform, Spark, Delta Lake, Lakeflow Jobs, Unity Catalog
Practice	Databricks Free Edition	Repeat the workshops and the final project on your own (`free_edition_guide.pdf`)
Accreditation	Databricks Fundamentals	Free self-paced course at Databricks Academy, followed by a short accreditation quiz
Next course	Databricks Explorer	The following course in the Altkom Akademia path
Certification	Data Engineer Associate, or Data Analyst Associate	Day-to-day data engineering; for analysts and BI, SQL analytics and dashboards
Advanced	Databricks Certified Data Engineer Professional	Production pipelines, performance and security
Specialisation	Machine learning, SQL analytics, platform administration	Role-specific learning paths

Databricks Academy (databricks.com/learn/training) offers free self-paced courses and practice exams for each stage.

PART 2

Certification

Data Engineer Associate

The natural next credential for data engineers after this course. It covers Delta Lake, Spark, the Databricks platform, the medallion architecture and basic ETL. Plan for this course plus a few months of hands-on work, and check the current exam guide on the Databricks certification page for the format and fee.

Data Analyst Associate

The equivalent credential for analysts and BI specialists. It covers Databricks SQL, managing data in the lakehouse, SQL queries, and dashboards and visualizations.

Associate Developer for Apache Spark

This certification focuses on the DataFrame API and Spark SQL rather than on the Databricks platform, which makes it most useful if you also work with Spark outside Databricks. Check the current exam guide for the format and fee.

Data Engineer Professional

Covers advanced Delta Lake, Lakeflow Spark Declarative Pipelines, Unity Catalog, performance tuning and security. The Associate certification is recommended first.

PART 3

Auto Loader

Auto Loader, the `cloudFiles` source, ingests files from cloud storage incrementally. It keeps track of the files it has already processed, so each run reads only new files instead of listing and scanning the whole directory. It supports ADLS Gen2, Amazon S3 and Google Cloud Storage, infers and evolves schemas, and is the standard way to ingest files in Lakeflow Spark Declarative Pipelines.

```
# Read new files incrementally
df = (spark.readStream
      .format("cloudFiles")
      .option("cloudFiles.format", "json")
      .option("cloudFiles.schemaLocation", "/Volumes/main/bronze/checkpoints/orders_schema")
      .load("/Volumes/main/landing/raw/orders/"))

# Append them to a Bronze table and stop once every available file is processed
(df.writeStream
   .option("checkpointLocation", "/Volumes/main/bronze/checkpoints/orders")
   .trigger(availableNow=True)
   .toTable("main.bronze.orders"))
```

Option	Purpose
`cloudFiles.format`	Source file format, such as `json`, `csv`, `parquet` or `avro`
`cloudFiles.schemaLocation`	Where the inferred schema is stored; use a Unity Catalog volume path
`cloudFiles.inferColumnTypes`	Infer column types for JSON and CSV instead of reading every column as a string
`cloudFiles.schemaEvolutionMode`	How new columns appearing in the source are handled
`cloudFiles.maxFilesPerTrigger`	Maximum number of files processed in one micro-batch

Documentation: docs.databricks.com/en/ingestion/cloud-object-storage/auto-loader/index.html

PART 4

Lakeflow Connect

Lakeflow Connect provides managed connectors that load data from databases, such as SQL Server, and from SaaS applications, such as Salesforce, Workday and ServiceNow, into Unity Catalog tables. You choose the source and the destination catalog and schema in the UI; Databricks runs the ingestion pipeline on serverless compute and loads changes incrementally.

Data source	Tool covered
A file uploaded by hand	The **Upload** dialog in Catalog Explorer (this course, M01)
Files in a volume or in cloud storage, read once	`spark.read`, `read_files()` (this course, M03)
Files that keep arriving in cloud storage	Auto Loader (Part 3)
A database or a SaaS application	Lakeflow Connect
An external database queried without copying the data	Lakehouse Federation

Documentation: learn.microsoft.com/azure/databricks/ingestion/lakeflow-connect/

PART 5

Lakeflow Spark Declarative Pipelines

Lakeflow Spark Declarative Pipelines, formerly Delta Live Tables, is the declarative framework for production ETL. You declare tables in Python or SQL, and Databricks determines the order of execution, processes data incrementally, applies data quality rules and records metrics for every update.

Creating a pipeline

1. In the workspace sidebar, open **Jobs & Pipelines** and create an ETL pipeline.
2. Name the pipeline, for example `retailhub_pipeline`, and add the source files.
3. Set Unity Catalog as the destination and choose the catalog and target schema.
4. Select serverless compute.

Pipeline code

The Python API is imported with `from pyspark import pipelines as dp`. Streaming tables are declared with `@dp.table` or `dp.create_streaming_table()`, and materialized views with `@dp.materialized_view`.

```
from pyspark import pipelines as dp
from pyspark.sql.functions import col, sum as sum_

# Bronze: raw orders, appended as new files arrive
dp.create_streaming_table("orders_bronze", comment="Raw orders, append-only")

@dp.append_flow(target="orders_bronze")
def orders_bronze_flow():
    return (spark.readStream
            .format("cloudFiles")
            .option("cloudFiles.format", "json")
            .option("cloudFiles.inferColumnTypes", "true")
            .load("/Volumes/main/landing/raw/orders/"))

# Silver: rows that break a rule are dropped and counted
dp.create_streaming_table(
    "orders_silver",
    comment="Validated orders",
    expect_all_or_drop={
        "valid_order_id": "order_id IS NOT NULL",
        "positive_amount": "total_amount > 0",
    },
)

@dp.append_flow(target="orders_silver")
def orders_silver_flow():
    return (spark.readStream.table("orders_bronze")
            .withColumn("order_date", col("order_datetime").cast("date"))
            .drop("order_datetime"))

# Gold: a materialized view refreshed on every pipeline update
@dp.materialized_view(name="revenue_by_day", comment="Daily revenue by payment method")
def revenue_by_day():
    return (spark.read.table("orders_silver")
            .groupBy("order_date", "payment_method")
            .agg(sum_("total_amount").alias("daily_revenue")))
```

Element	Purpose
`dp.create_streaming_table()`	Declares a streaming table that one or more flows write to
`@dp.append_flow(target=...)`	Defines a query that appends data to a streaming table
`@dp.materialized_view`	Declares a materialized view built from a batch query
`expect_all_or_drop`	Drops rows that break a rule and records how many were dropped
`expect_all`	Keeps every row and records rule violations as metrics
`expect_all_or_fail`	Stops the update when any row breaks a rule
`spark.readStream.table()`	Reads another pipeline table incrementally
`spark.read.table()`	Reads another pipeline table in full

Running and monitoring

An update can refresh tables incrementally, processing only new data, or run a full refresh that rebuilds them. Pipelines can be scheduled like any job. While an update runs, the pipeline graph shows the state of each table; afterwards, the data quality view shows for every rule how many rows passed and how many were dropped.

Rows removed by `expect_all_or_drop` are not kept anywhere. If you need to inspect them, write the failing rows to a separate table yourself, using the inverse of the rule.

Pipelines compared with notebook jobs

	Notebooks in Lakeflow Jobs	Lakeflow Spark Declarative Pipelines
Execution order	Task dependencies defined by hand	Derived from the references between tables
Data quality	Custom checks in code	Declared expectations with built-in metrics
Incremental processing	Implemented by hand	Provided by the framework
After a failure	Repair run of the failed tasks	Automatic retries
Monitoring	Task logs and output	Pipeline graph and data quality metrics

Documentation: docs.databricks.com/en/ldp/

PART 6

Unity Catalog in depth

The course used Unity Catalog for the three-level namespace and basic privileges. It also provides finer-grained controls.

Row filters

A row filter is a SQL function that returns true for the rows a user may see. It is attached to a table and evaluated for every query.

```
CREATE FUNCTION main.security.region_filter(region STRING)
RETURN is_account_group_member('sales_admins') OR region = 'EMEA';

ALTER TABLE main.sales.transactions
SET ROW FILTER main.security.region_filter ON (region);
```

Column masks

A column mask is a SQL function that returns the value a user sees in place of the stored one.

```
CREATE FUNCTION main.security.email_mask(email STRING)
RETURN CASE
         WHEN is_account_group_member('data_admins') THEN email
         ELSE regexp_replace(email, '^[^@]+', '***')
       END;

ALTER TABLE main.retail.customers
ALTER COLUMN email SET MASK main.security.email_mask;
```

Lineage

Unity Catalog records table-level and column-level lineage for queries, notebooks and jobs without any configuration. Lineage is shown on the Lineage tab of a table in Catalog Explorer and can be queried in `system.access.table_lineage`.

Tags

```
ALTER TABLE main.retail.customers SET TAGS ('pii' = 'true', 'domain' = 'customer');

ALTER TABLE main.retail.customers ALTER COLUMN email SET TAGS ('sensitivity' = 'high');
```

OpenSharing

OpenSharing (formerly Delta Sharing) gives other organisations read-only access to live tables without copying data. Recipients do not need a Databricks workspace; they can read shares with open clients for Python, Spark, pandas or Power BI.

Documentation: docs.databricks.com/en/data-governance/unity-catalog/index.html

PART 7

Structured Streaming

Auto Loader is built on Structured Streaming, which processes an unbounded stream of data as a series of small batches.

```
from pyspark.sql.functions import from_json, col, window, sum as sum_
from pyspark.sql.types import StructType, StructField, StringType, DoubleType, TimestampType

schema = StructType([
    StructField("order_id", StringType()),
    StructField("payment_method", StringType()),
    StructField("total_amount", DoubleType()),
    StructField("event_time", TimestampType()),
])

orders = (spark.readStream
          .format("kafka")
          .option("kafka.bootstrap.servers", "broker:9092")
          .option("subscribe", "orders")
          .load()
          .select(from_json(col("value").cast("string"), schema).alias("o"))
          .select("o.*"))

revenue = (orders
           .withWatermark("event_time", "10 minutes")
           .groupBy(window("event_time", "5 minutes"), "payment_method")
           .agg(sum_("total_amount").alias("revenue")))

(revenue.writeStream
    .outputMode("append")
    .option("checkpointLocation", "/Volumes/main/gold/checkpoints/revenue_5min")
    .toTable("main.gold.revenue_5min"))
```

Trigger	Behaviour
`processingTime="1 minute"`	Starts a micro-batch at a fixed interval
`availableNow=True`	Processes everything available, then stops
`once=True`	Deprecated; use `availableNow` instead
`continuous="1 second"`	Experimental low-latency processing for a limited set of operations

Documentation: docs.databricks.com/en/structured-streaming/index.html

PART 8

Performance

Liquid clustering and predictive optimization

Liquid clustering is the recommended data layout for new tables and replaces both partitioning and Z-ordering. Clustering keys can be changed without rewriting existing data. It is generally available for Delta tables from Databricks Runtime 15.4 LTS.

```
CREATE TABLE main.silver.orders
CLUSTER BY (customer_id, order_date)
AS SELECT * FROM main.bronze.orders;

-- Cluster new and changed data; the operation is incremental
OPTIMIZE main.silver.orders;

-- Let Databricks choose and adapt the keys (Unity Catalog managed tables)
ALTER TABLE main.silver.orders CLUSTER BY AUTO;
```

Predictive optimization runs `OPTIMIZE`, `VACUUM` and `ANALYZE` automatically on Unity Catalog managed tables, so you do not schedule table maintenance yourself. It is enabled by default for accounts created since November 2024 and is being rolled out to older accounts; the documentation explains how to check whether it is on.

OPTIMIZE and Z-ordering on older tables

On tables without liquid clustering, `OPTIMIZE` compacts small files and `ZORDER BY` co-locates related values. Z-ordering helps most on columns that are filtered often and have many distinct values.

```
OPTIMIZE main.silver.customers;
OPTIMIZE main.silver.transactions ZORDER BY (customer_id);
```

Photon

Photon is a vectorised query engine written in C++. It runs by default on SQL warehouses and serverless compute, and can be enabled on classic clusters. It speeds up SQL and DataFrame workloads such as scans, joins and aggregations without code changes.

Adaptive query execution

AQE is enabled by default. It re-plans queries at run time, switching join strategies, merging small shuffle partitions and handling skewed data.

PART 9

MLflow

MLflow tracks machine learning experiments and is built into every Databricks workspace.

```
import mlflow
from sklearn.ensemble import RandomForestClassifier

with mlflow.start_run(run_name="rf_baseline"):
    model = RandomForestClassifier(n_estimators=100, max_depth=5)
    model.fit(X_train, y_train)

    mlflow.log_param("n_estimators", 100)
    mlflow.log_param("max_depth", 5)
    mlflow.log_metric("accuracy", model.score(X_test, y_test))
    mlflow.sklearn.log_model(model, "model")
```

Runs are listed under **Experiments** in the workspace sidebar. MLflow supports scikit-learn, XGBoost, PyTorch, TensorFlow, Hugging Face Transformers and LangChain, among others.

Documentation: docs.databricks.com/en/mlflow/index.html

PART 10

Advanced orchestration

Lakeflow Jobs support much more than a sequence of notebooks.

Feature	Use
Task dependencies	A graph of tasks, including tasks that run in parallel
If/else conditions	Choosing the next task based on a value returned by an earlier task
For each	Running a task once for every element of a list
Task values	Passing small results between tasks with `dbutils.jobs.taskValues`
Repair runs	Rerunning only the failed tasks and the tasks that depend on them
Notifications	Email, Slack, Microsoft Teams or webhooks on start, success or failure

```
# Upstream task: publish a value
dbutils.jobs.taskValues.set(key="rows_processed", value=15000)

# Downstream task: read it
rows = dbutils.jobs.taskValues.get(taskKey="ingest", key="rows_processed", default=0)
```

PART 11

Integrations

Tool	How it connects
Power BI	Databricks connector to a SQL warehouse, or Partner Connect
Tableau	Databricks connector, or ODBC and JDBC to a SQL warehouse
Azure Data Factory	Linked service with a Databricks notebook or job activity
dbt	The `dbt-databricks` adapter
Visual Studio Code	Databricks extension, which runs code on Databricks compute from the IDE
Great Expectations	Python library for data quality checks in notebooks

Resources

Resource	Address
Databricks documentation	docs.databricks.com
Databricks Academy	databricks.com/learn/training
Community forums	community.databricks.com
Databricks blog	databricks.com/blog
Delta Lake on GitHub	github.com/delta-io/delta